FE630_final_project_file

December 18, 2024

1 Portfolio Optimization and Strategy Analysis

1.1 Overview:

This notebook performs portfolio optimization and strategy analysis using a set of diversified ETFs and the Fama-French 3-factor model. It evaluates **long-only** and **long-short** strategies under various market conditions and risk aversion levels (λ). The analysis is aimed at understanding portfolio behavior across different market regimes and comparing performance with a benchmark index (SPY).

1.2 Key Features:

1. **Data Preparation**:
   - Downloads historical ETF price data and Fama-French factor data.
   - Computes daily returns and excess returns (over risk-free rate).
2. **Portfolio Optimization**:
   - Optimizes weights based on expected returns, covariance matrix, and beta constraints.
   - Supports both long-only and long-short strategies.
3. **Backtesting**:
   - Simulates portfolio performance with weekly rebalancing.
   - Tracks portfolio returns, weights, and cumulative performance for multiple risk aversion values (λ).
4. **Market Regime Analysis**:
   - Evaluates performance during key market regimes:
     - Subprime Crisis
     - Post-Subprime Recovery
     - COVID Era
     - Post-COVID Recovery
   - Normalizes regime performance for easy comparison.
5. **Comparison with SPY**:
   - Benchmarks strategy performance against SPY over the entire analysis period.
6. **Additional Metrics and Visualizations**:
   - Rolling volatility (30-day window).
   - Sharpe ratio comparisons.
   - Heatmap of portfolio weights over time.

1.3 How to Use:

- **Understand the Workflow**:

  - Each block in the notebook is clearly labeled and explained to help you follow the workflow step-by-step.
- **Modify Inputs**:
  - You can change ETFs, date ranges, and risk aversion values (λ) as needed to customize the analysis.
- **Extend Analysis**:
  - Add more factors, alternative benchmarks, or customize portfolio constraints to suit your requirements.

1.4 Expected Output:

- Cumulative return plots for different strategies and λ values.
- Strategy performance during specific market regimes.
- Benchmark comparison with SPY.
- Rolling volatility, Sharpe ratios, and portfolio weights visualizations.

1.5 Potential Improvements:

- Include transaction costs to account for real-world trading frictions.
- Experiment with alternative factor models or data sources.
- Integrate advanced optimization methods like reinforcement learning or machine learning-based factor modeling.

$$\text{Objective Function: } \max\left(\text{Portfolio Return} - \lambda \times \text{Portfolio Risk}\right)$$

This notebook is designed for anyone interested in portfolio optimization, quantitative finance, or backtesting investment strategies. The code and visualizations aim to provide actionable insights and can be easily adapted for other datasets or use cases.

2 Block 1: Importing Required Libraries

This block imports all the necessary libraries for data downloading, manipulation, optimization, and visualization.

- **Key Libraries**:
  - `yfinance`: For downloading historical ETF price data.
  - `pandas` and `numpy`: For data manipulation and numerical operations.
  - `matplotlib`: For visualizing results.
  - `statsmodels.api`: For running Fama-French regressions.
  - `cvxpy`: For portfolio optimization using constrained convex programming.
- **Potential Changes**:
  - If better visualization is required, libraries like `seaborn` or `plotly` can be added.
  - Replace `cvxpy` solver (`SCS`) with faster solvers like `OSQP` if performance is an issue.

[42]:
```
# Import necessary libraries
import yfinance as yf
import pandas as pd
import numpy as np
```

```
import matplotlib.pyplot as plt
import statsmodels.api as sm
import cvxpy as cp
from datetime import timedelta
import warnings
from tqdm import tqdm

warnings.filterwarnings("ignore")

# Global Variables
etfs = ['FXE', 'EWJ', 'GLD', 'QQQ', 'SPY', 'SHV', 'DBA', 'USO', 'XBI', 'ILF',␣
↪'EPP', 'FEZ']
start_date = '2007-03-01'
end_date = '2024-03-31'
initial_capital = 1_000_000
```

3 Block 2: Define Global Variables and Download Data

This block initializes global variables such as: - ETF tickers: A diversified list of asset classes to build the portfolio. - Start and end dates: Defines the analysis period. - Initial capital: Sets the starting value of the portfolio.

It also: 1. Downloads daily adjusted closing prices for the ETFs. 2. Computes daily returns for all ETFs. 3. Downloads Fama-French 3-factor data for risk-adjusted returns calculation.

- **Potential Changes**:
  - Update the ETF list or time range based on new requirements.
  - Add custom datasets or alternative risk factors beyond Fama-French (e.g., momentum or volatility factors).
  - Clip extreme values in returns if outliers impact the analysis.

[2]:
```
# Load ETF data
etf_data = yf.download(etfs, start=start_date, end=end_date)['Adj Close']
daily_returns = etf_data.pct_change().dropna()

# Load Fama-French 3-factor data
fama_french_url = "http://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/
↪F-F_Research_Data_Factors_daily_CSV.zip"
fama_french_data = pd.read_csv(fama_french_url, skiprows=3, skipfooter=3,␣
↪engine='python')
fama_french_data.columns = ['Date', 'Mkt-RF', 'SMB', 'HML', 'RF']
fama_french_data['Date'] = pd.to_datetime(fama_french_data['Date'].astype(str),␣
↪format='%Y%m%d')
fama_french_data.set_index('Date', inplace=True)
fama_french_data = fama_french_data.loc[start_date:end_date] / 100

# Compute excess returns
```

```
excess_returns = daily_returns.subtract(fama_french_data['RF'], axis=0)
```

```
[*********************100%%**********************]  12 of 12 completed
```

4 Block 3: Factor Loadings Calculation

This block defines the function `compute_factor_loadings` to calculate factor loadings for each ETF using the Fama-French 3-factor model.

- **Process**:
  - Aligns excess returns and Fama-French factor data.
  - Fits an Ordinary Least Squares (OLS) regression for each ETF.
  - Outputs the regression coefficients (loadings) for each factor: `Mkt-RF`, `SMB`, `HML`.
- **Potential Changes**:
  - Replace Fama-French factors with alternative factor models (e.g., Carhart 4-factor, AQR's Q-factor model).
  - Add statistical diagnostics (e.g., R-squared) to evaluate model fit.

[3]:
```
def compute_factor_loadings(excess_returns, ff_data):
    """
    Compute factor loadings for each asset using Fama-French 3-factor model.
    """
    factor_loadings = {}
    for asset in excess_returns.columns:
        aligned_data = pd.concat([excess_returns[asset], ff_data], axis=1).
↪dropna()
        X = sm.add_constant(aligned_data[['Mkt-RF', 'SMB', 'HML']])
        y = aligned_data[asset]
        model = sm.OLS(y, X).fit()
        factor_loadings[asset] = model.params.values
    return pd.DataFrame(factor_loadings).T
```

5 Block 4: Portfolio Optimization

This block defines the function `optimize_portfolio` for optimizing portfolio weights using: 1. Expected returns. 2. Covariance matrix. 3. Beta constraints based on Fama-French factor loadings.

- **Optimization Details**:
  - Long-only strategy: Constrains weights to [0, 1].
  - Long-short strategy: Allows weights between [-2, 2].
  - The objective function maximizes risk-adjusted returns using risk aversion ((λ)).
- **Potential Changes**:
  - Modify beta bounds or weight limits for different strategies.
  - Replace the convex optimization framework with machine learning-based portfolio construction (e.g., reinforcement learning).
  - Improve performance by exploring faster solvers or adjusting constraints.

[37]:
```python
# Block 4: Portfolio Optimization
# This block defines a function to optimize portfolio weights using the
# expected returns, covariance matrix, and beta constraints.
# Additional constraints ensure weights are capped to avoid excessive␣
↪concentration.

def optimize_portfolio(expected_returns, covariance_matrix, beta, strategy_type,␣
↪risk_aversion):
    """
    Optimize portfolio weights based on expected returns, covariance, and beta.
    Adjusts beta constraints and limits individual weights for concentration␣
↪control.
    """
    n = len(expected_returns)
    weights = cp.Variable(n)

    # Regularize covariance matrix
    covariance_matrix += np.eye(covariance_matrix.shape[0]) * 1e-6

    # Set beta constraints based on strategy type
    if strategy_type == "long-only":
        beta_bounds = [0.8, 1.2]  # Tighter beta range for long-only strategy
        weight_limit = 0.7  # Individual weight limit for long-only strategy
    elif strategy_type == "long-short":
        beta_bounds = [-0.5, 0.5]  # Wider beta range for long-short strategy
        weight_limit = 0.5  # Individual weight limit for long-short strategy

    # Define constraints
    constraints = [
        cp.sum(weights) == 1,  # Fully invested portfolio
        beta_bounds[0] <= beta @ weights,
        beta @ weights <= beta_bounds[1],
        cp.abs(weights) <= weight_limit  # Cap absolute value of weights to␣
↪avoid concentration
    ]

    if strategy_type == "long-only":
        constraints += [weights >= 0, weights <= 1]  # Long-only constraints

    # Define the optimization problem
    portfolio_return = expected_returns @ weights
    portfolio_risk = cp.quad_form(weights, covariance_matrix)
    objective = cp.Maximize(portfolio_return - risk_aversion * portfolio_risk)
    problem = cp.Problem(objective, constraints)
    problem.solve(solver=cp.SCS)

    # Check if optimization was successful
```

```
    if problem.status != cp.OPTIMAL:
        raise ValueError(f"Optimization failed with status: {problem.status}")

    return weights.value
```

6 Block 5: Backtesting Setup

This block performs backtesting for both `long-only` and `long-short` strategies across multiple lambda values ((λ= 0.1, 0.5, 1.0)).

- **Key Features**:
  - Uses a rolling lookback window of 120 days for rebalancing.
  - Records portfolio weights, returns, and cumulative portfolio values for each strategy and lambda combination.
- **Potential Changes**:
  - Experiment with different rebalancing frequencies (e.g., monthly or daily).
  - Vary lookback window sizes to study their impact on performance.
  - Integrate transaction cost modeling for more realistic backtesting results.

[22]:
```
strategies = ['long-only', 'long-short']
lambdas = [0.1, 0.5, 1.0]
results = {strategy: {lambda_val: [] for lambda_val in lambdas} for strategy in␣
 ↪strategies}
portfolio_history = {strategy: {lambda_val: [] for lambda_val in lambdas} for␣
 ↪strategy in strategies}
weights_history = {strategy: {lambda_val: [] for lambda_val in lambdas} for␣
 ↪strategy in strategies}

rebalancing_dates = pd.date_range(start=start_date, end=end_date, freq='W')
beta_bounds = [0, 1]

for strategy in strategies:
    for risk_aversion in lambdas:
        portfolio_value = 1  # Normalized initial value for cumulative returns
        history = []
        weights_tracker = []
        for date in rebalancing_dates:
            if date not in excess_returns.index:
                date = excess_returns.index[excess_returns.index.get_loc(date,␣
 ↪method='nearest')]

            lookback_period = 120
            start_period = date - timedelta(days=lookback_period)

            if start_period < excess_returns.index[0]:
                continue
```

```
            # Compute factor loadings and optimize portfolio
            factor_loadings = compute_factor_loadings(excess_returns.
↪loc[start_period:date], fama_french_data.loc[start_period:date])
            expected_returns = factor_loadings[0].values
            covariance_matrix = excess_returns.loc[start_period:date].cov().
↪values
            beta = factor_loadings[1].values

            # Optimize weights
            weights = optimize_portfolio(expected_returns, covariance_matrix,␣
↪beta, strategy, risk_aversion)

            # Update weights history
            weights_tracker.append(weights)

            # Calculate portfolio return and update value
            portfolio_return = np.dot(weights, daily_returns.loc[date].values)
            portfolio_value *= (1 + portfolio_return)

            # Append to history
            history.append((date, portfolio_value))

        # Store results for this strategy and lambda
        portfolio_history[strategy][risk_aversion] = pd.DataFrame(history,␣
↪columns=['Date', 'Portfolio Value'])
        weights_history[strategy][risk_aversion] = pd.DataFrame(weights_tracker,␣
↪columns=etfs, index=rebalancing_dates[:len(weights_tracker)])
```

7 Block 6: Plotting Performance Across Regimes

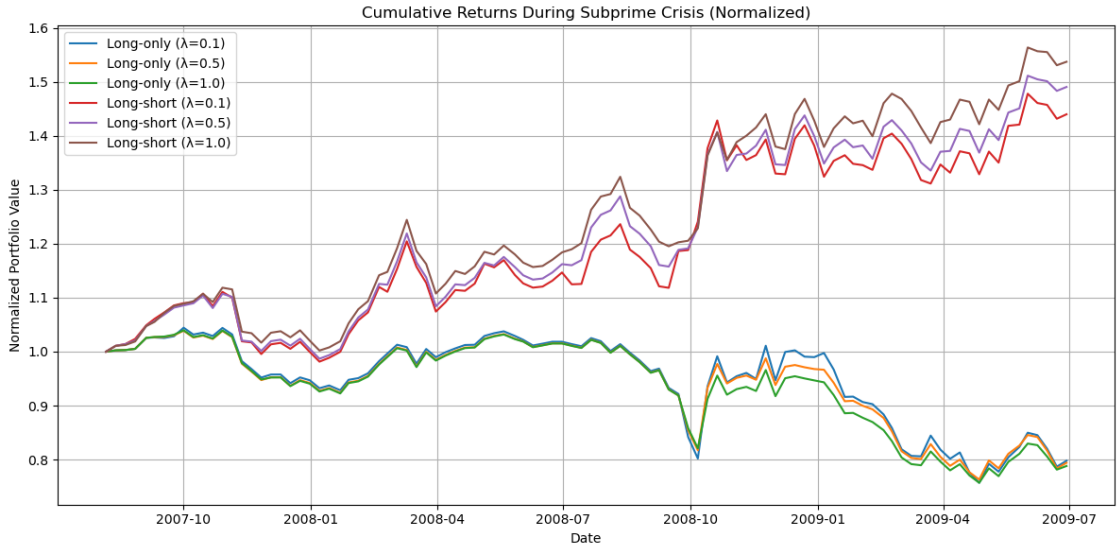
This block analyzes portfolio performance during predefined market regimes: 1. Subprime Crisis (2007–2009). 2. Post-Subprime Recovery (2009–2019). 3. COVID Era (2020–2021). 4. Post-COVID Recovery (2022–2024).

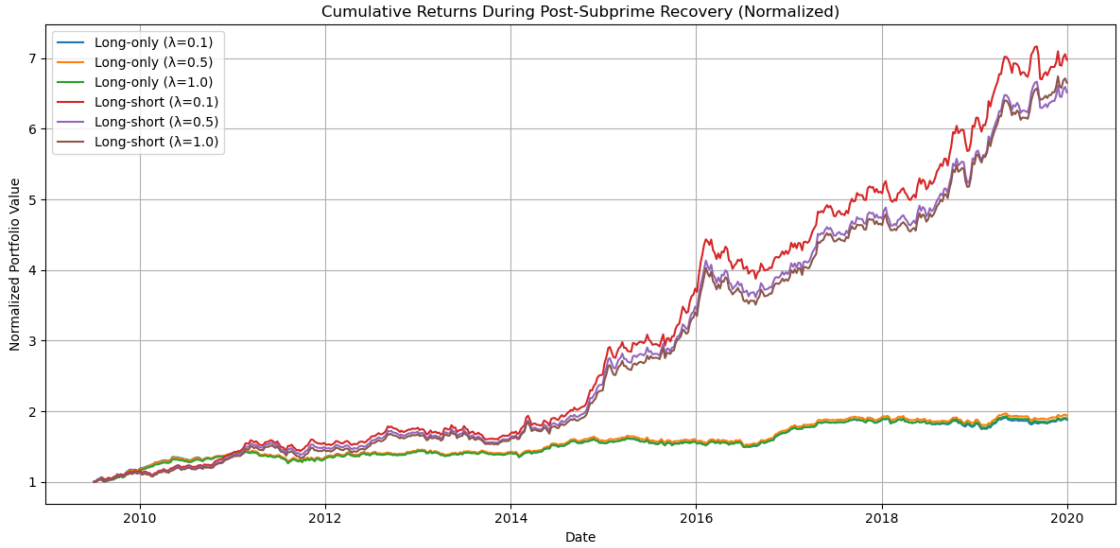
- **Key Features**:
  - Normalizes portfolio values to start at 1 for each regime, enabling fair comparisons.
  - Visualizes cumulative returns for all strategies and lambda values within each regime.
- **Potential Changes**:
  - Adjust regime definitions based on new data or research focus.
  - Include benchmarks (e.g., SPY) in regime analysis for additional context.

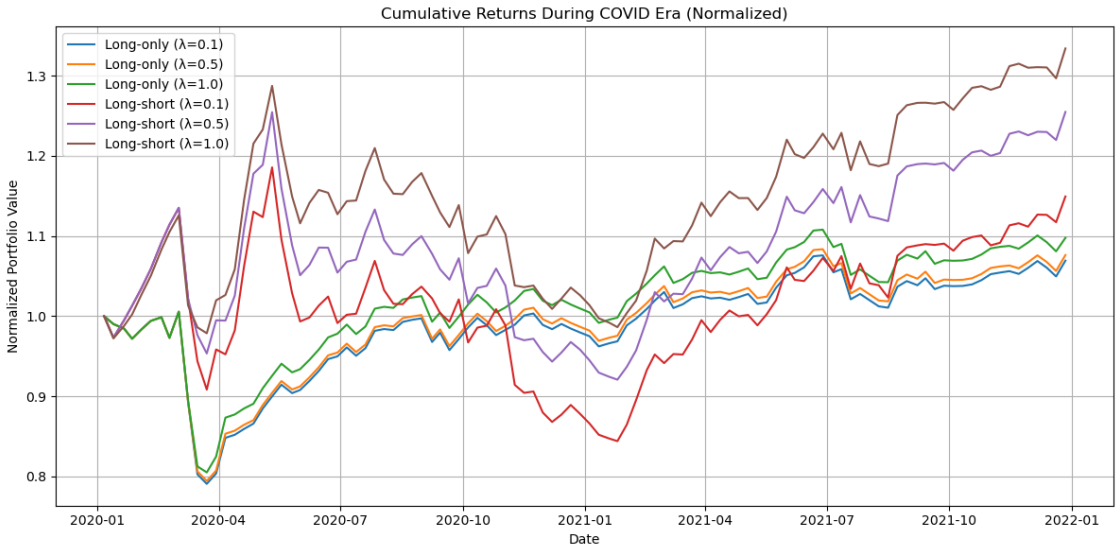
[23]:
```
def define_regimes():
    """
    Define the time periods for different market regimes.
    """
    return {
        "Subprime Crisis": ("2007-08-01", "2009-06-30"),
```

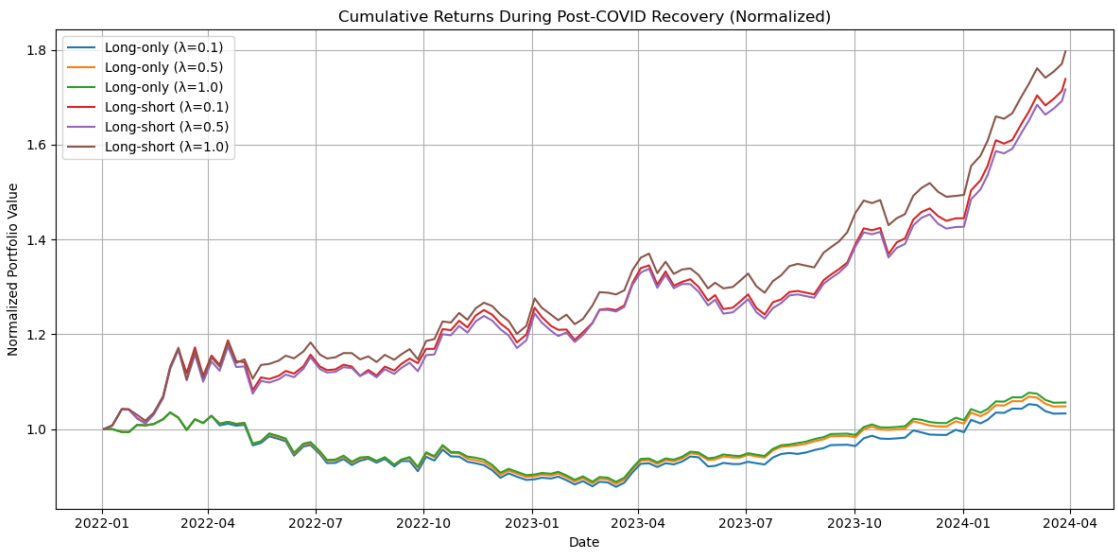
```
        "Post-Subprime Recovery": ("2009-07-01", "2019-12-31"),
        "COVID Era": ("2020-01-01", "2021-12-31"),
        "Post-COVID Recovery": ("2022-01-01", "2024-03-31"),
    }

def normalize_regime_data(history_df, start_period, end_period):
    """
    Normalize portfolio values within a specific regime to ensure they start at␣
↪1.
    """
    regime_data = history_df[(history_df['Date'] >= start_period) &␣
↪(history_df['Date'] <= end_period)]
    if not regime_data.empty:
        normalized_values = regime_data['Portfolio Value'] /␣
↪regime_data['Portfolio Value'].iloc[0]
        regime_data = regime_data.copy()
        regime_data['Normalized Value'] = normalized_values
    return regime_data

def plot_regime_comparison_normalized(portfolio_history, regimes):
    """
    Plot cumulative returns for different market regimes with normalization.
    """
    for regime_name, (start_period, end_period) in regimes.items():
        plt.figure(figsize=(12, 6))
        for strategy in strategies:
            for risk_aversion in lambdas:
                history_df = portfolio_history[strategy][risk_aversion]
                regime_data = normalize_regime_data(history_df, start_period,␣
↪end_period)
                if not regime_data.empty:
                    plt.plot(
                        regime_data['Date'],
                        regime_data['Normalized Value'],
                        label=f'{strategy.capitalize()} (λ={risk_aversion})'
                    )

        # Customize plot
        plt.title(f'Cumulative Returns During {regime_name} (Normalized)')
        plt.xlabel('Date')
        plt.ylabel('Normalized Portfolio Value')
        plt.legend()
        plt.grid()
        plt.tight_layout()
        plt.show()

# Execute the regime analysis
```

```
regimes = define_regimes()
plot_regime_comparison_normalized(portfolio_history, regimes)
```









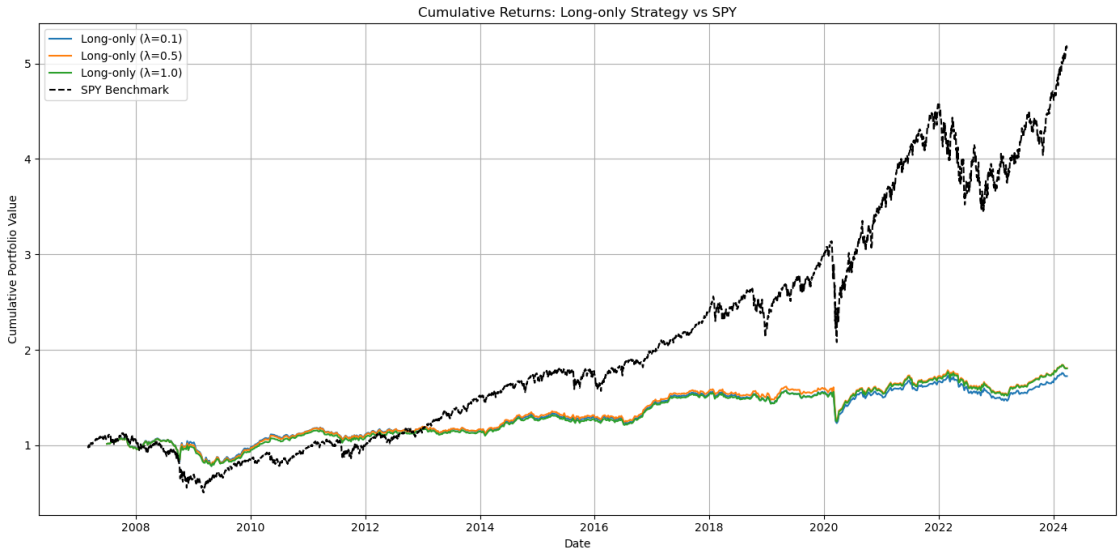
8 Block 7: Comparing Strategies vs SPY

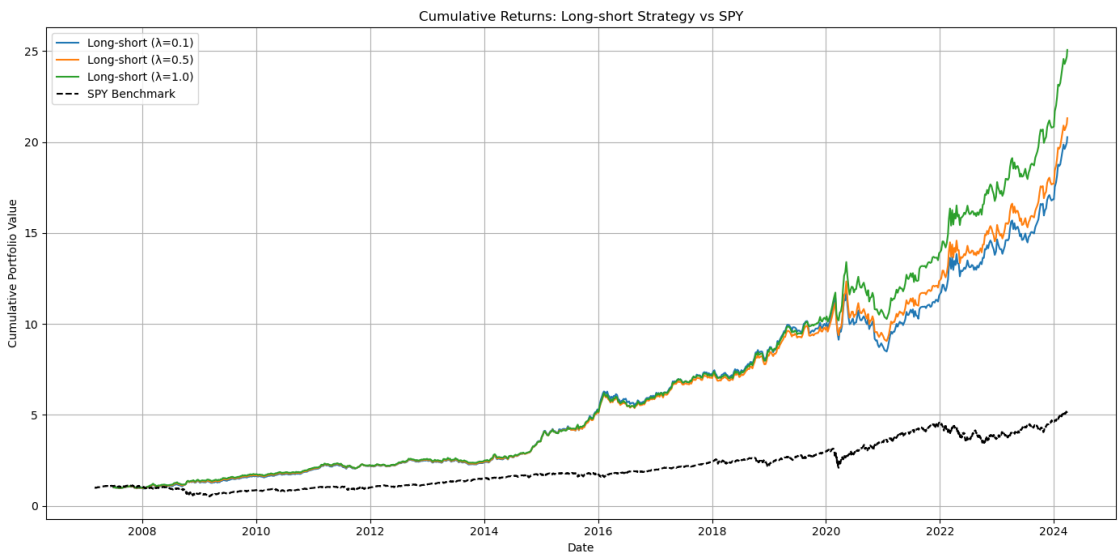
This block compares the cumulative returns of each strategy (long-only and long-short) with the SPY benchmark over the full timeframe.

- **Key Features**:
  - Plots cumulative portfolio value for all lambda values alongside SPY.
  - Provides separate visualizations for each strategy for clarity.
- **Potential Changes**:
  - Add other benchmarks (e.g., QQQ or other broad market indices).

– Explore drawdown metrics for a more comprehensive comparison.

[24]:
```
def plot_strategy_vs_spy(portfolio_history, spy_returns, strategies, lambdas):
    """
    Plot cumulative returns of each strategy and compare with SPY benchmark.
    """
    # Compute SPY cumulative returns
    spy_cumulative = (1 + spy_returns).cumprod()

    for strategy in strategies:
        plt.figure(figsize=(14, 7))
        for risk_aversion in lambdas:
            # Extract portfolio data
            history_df = portfolio_history[strategy][risk_aversion]
            cumulative_returns = history_df['Portfolio Value']

            # Plot strategy cumulative returns
            plt.plot(history_df['Date'], cumulative_returns, label=f'{strategy.
 ↪capitalize()} (λ={risk_aversion})')

        # Plot SPY benchmark
        plt.plot(spy_cumulative.index, spy_cumulative.values, label='SPY␣
 ↪Benchmark', linestyle='--', color='black')

        # Customize plot
        plt.title(f'Cumulative Returns: {strategy.capitalize()} Strategy vs SPY')
        plt.xlabel('Date')
        plt.ylabel('Cumulative Portfolio Value')
        plt.legend()
        plt.grid()
        plt.tight_layout()
        plt.show()

# Compute SPY returns
spy_returns = daily_returns['SPY']

# Execute the plot function
plot_strategy_vs_spy(portfolio_history, spy_returns, strategies, lambdas)
```





9 Block 8: Additional Strategy Analysis and Visualization

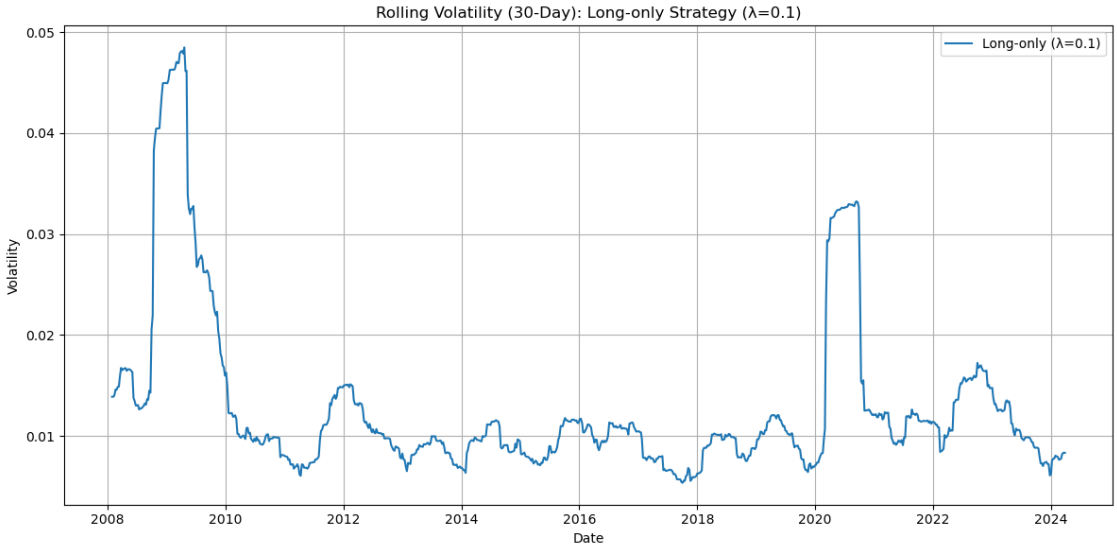
This block provides in-depth analysis and visualizations to evaluate strategy performance, including: 1. **Rolling Volatility**: - Computes and plots 30-day rolling volatility for the selected lambda ((λ= 0.1)). 2. **Sharpe Ratio Comparison**: - Compares Sharpe ratios across lambda values for both strategies. 3. **Portfolio Weights Heatmap**: - Displays a heatmap of portfolio weights over time for all strategies and lambda values.

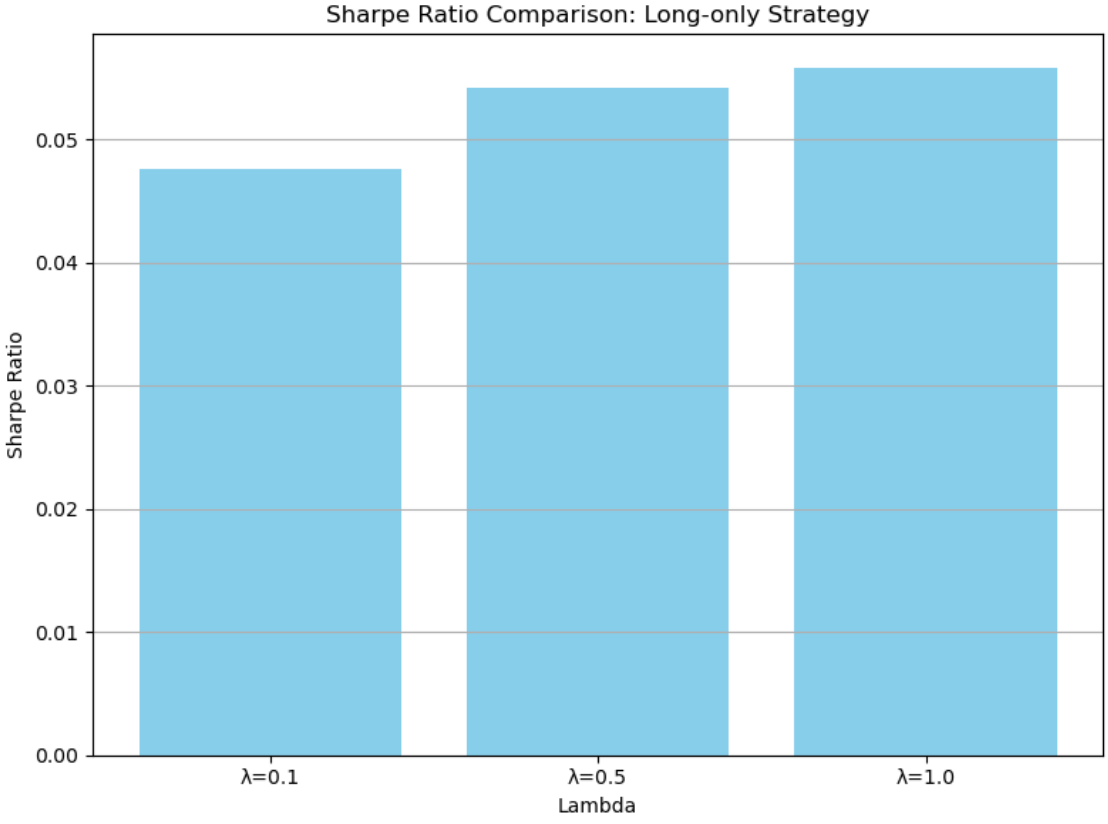
- **Potential Changes**:
  - Experiment with different rolling windows for volatility analysis.

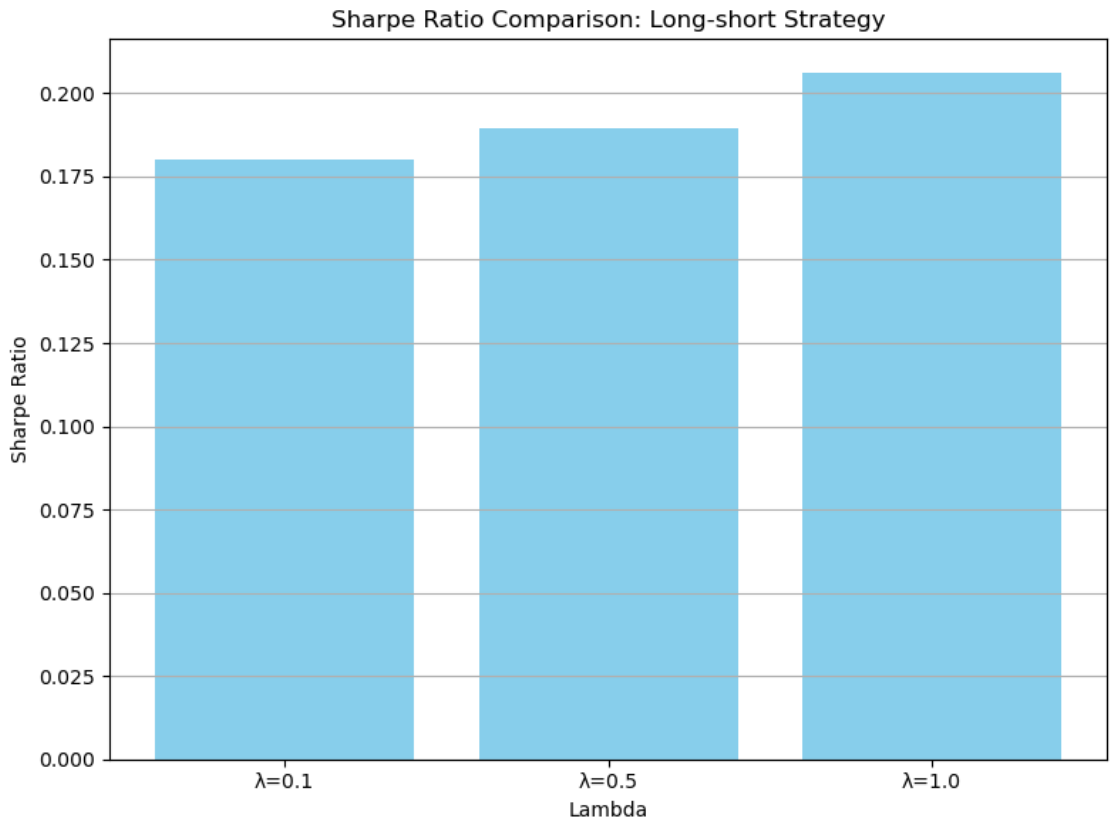
- Extend the heatmap to include transaction costs or turnover metrics.
- Add Treynor Ratio or Sortino Ratio for further risk-adjusted performance evaluation.

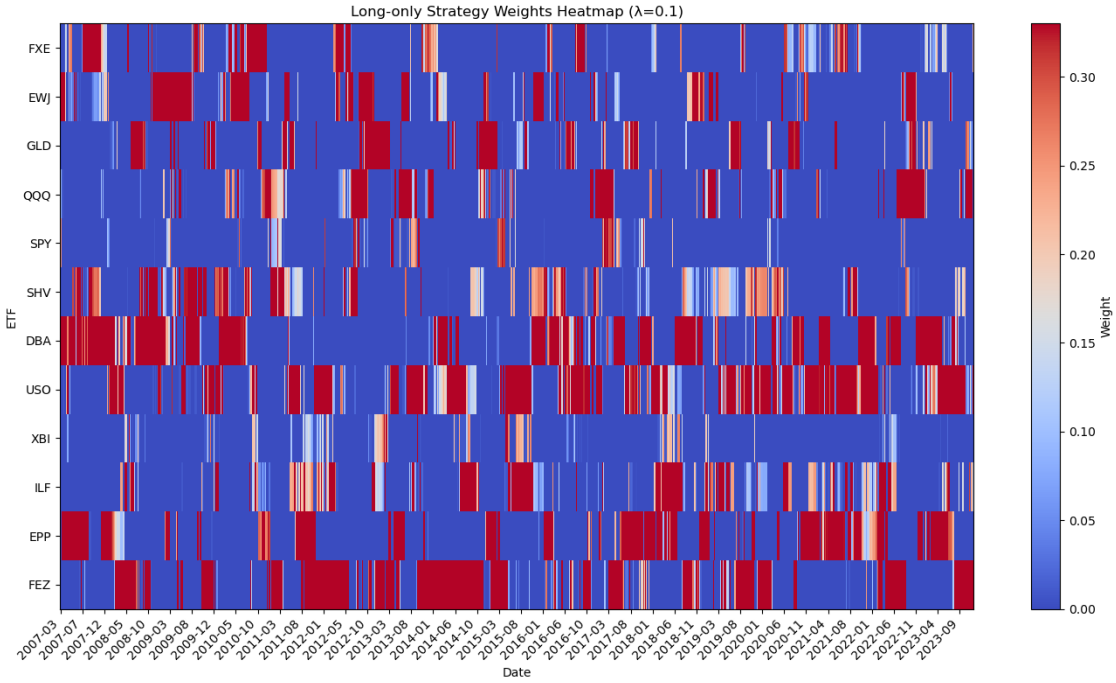
[25]:
```
# 1. Rolling Volatility Plot
def plot_rolling_volatility(portfolio_history,␣
↪lambda_for_rolling_vol,strategies):
    """
    Plot rolling 30-day volatility for the selected lambda value.
    """
    for strategy in strategies:
        for risk_aversion in lambda_for_rolling_vol:
            history_df = portfolio_history[strategy][risk_aversion]
            history_df['Return'] = history_df['Portfolio Value'].pct_change()
            rolling_vol = history_df['Return'].rolling(window=30).std()
            plt.figure(figsize=(12, 6))
            plt.plot(history_df['Date'], rolling_vol, label=f'{strategy.
↪capitalize()} (λ={risk_aversion})')
            plt.title(f'Rolling Volatility (30-Day): {strategy.capitalize()}␣
↪Strategy (λ={risk_aversion})')
            plt.xlabel('Date')
            plt.ylabel('Volatility')
            plt.legend()
            plt.grid()
            plt.tight_layout()
            plt.show()

# Execute rolling volatility plot for λ=0.1
lambda_for_rolling_vol = [0.1]
strategy_for_rolling_vol = ['long-only']
plot_rolling_volatility(portfolio_history,␣
↪lambda_for_rolling_vol,strategy_for_rolling_vol)

# 2. Sharpe Ratio Comparison
def plot_sharpe_ratio_comparison(portfolio_history, lambdas):
    """
    Plot Sharpe ratio comparisons across different lambda values for each␣
↪strategy.
    """
    for strategy in strategies:
        sharpe_ratios = []
        for risk_aversion in lambdas:
            history_df = portfolio_history[strategy][risk_aversion]
            mean_return = history_df['Portfolio Value'].pct_change().mean()
            std_return = history_df['Portfolio Value'].pct_change().std()
            sharpe_ratios.append(mean_return / std_return)
        plt.figure(figsize=(8, 6))
        plt.bar([f'λ={l}' for l in lambdas], sharpe_ratios, color='skyblue')
```

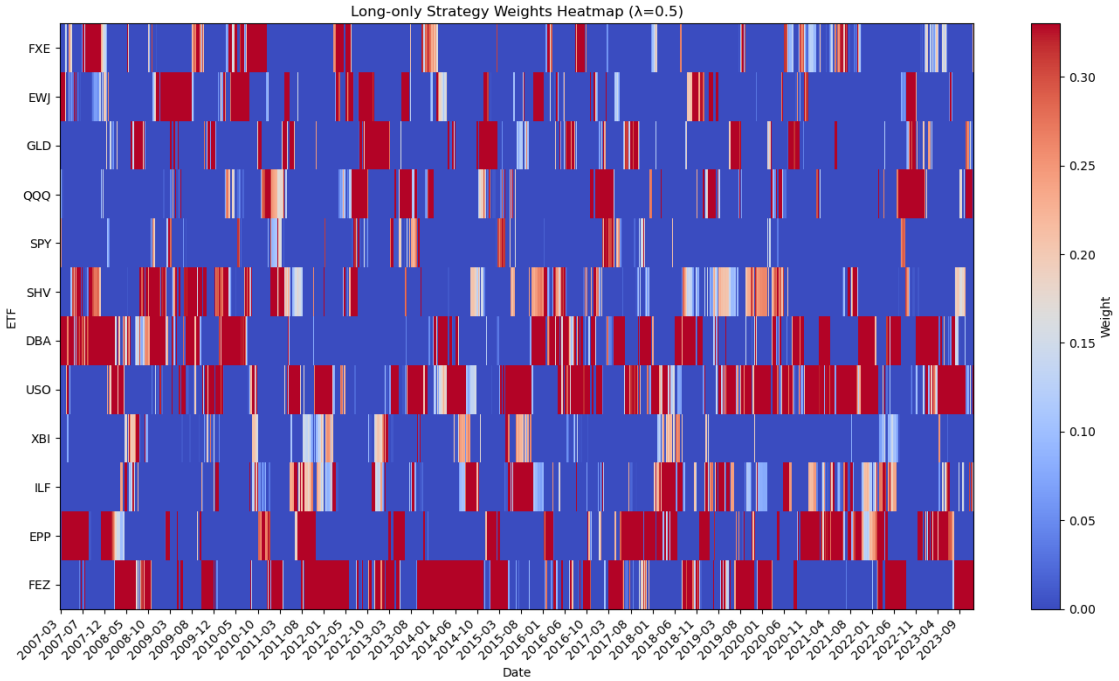
```
        plt.title(f'Sharpe Ratio Comparison: {strategy.capitalize()} Strategy')
        plt.ylabel('Sharpe Ratio')
        plt.xlabel('Lambda')
        plt.grid(axis='y')
        plt.tight_layout()
        plt.show()

# Execute Sharpe ratio comparison
plot_sharpe_ratio_comparison(portfolio_history, lambdas)

# 3. Portfolio Weights Heatmap
def plot_weights_heatmap(weights_history):
    """
    Plot heatmap of portfolio weights for each strategy and lambda value.
    """
    for strategy in strategies:
        for risk_aversion in lambdas:
            weights_df = weights_history[strategy][risk_aversion]
            plt.figure(figsize=(14, 8))
            plt.imshow(weights_df.T, aspect='auto', cmap='coolwarm',␣
 ↪interpolation='nearest')
            plt.colorbar(label='Weight')
            skip_interval = 21
            xticks_positions = np.arange(0, len(weights_df), skip_interval)
            xticks_labels = weights_df.index[::skip_interval].strftime('%Y-%m')
            plt.xticks(ticks=xticks_positions, labels=xticks_labels,␣
 ↪rotation=45, ha='right')
            plt.yticks(ticks=np.arange(len(weights_df.columns)),␣
 ↪labels=weights_df.columns)
            plt.title(f'{strategy.capitalize()} Strategy Weights Heatmap␣
 ↪(λ={risk_aversion})')
            plt.xlabel('Date')
            plt.ylabel('ETF')
            plt.tight_layout()
            plt.show()

# Execute weights heatmap
plot_weights_heatmap(weights_history)
```

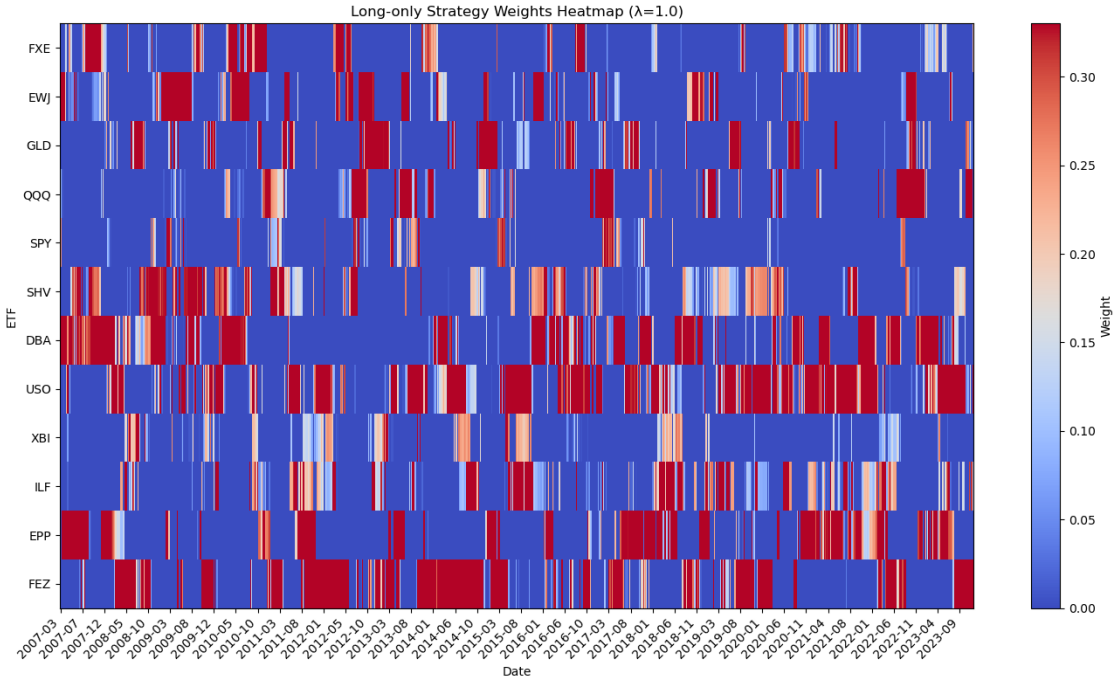

Rolling Volatility (30-Day): Long-only Strategy (λ=0.1)
Long-only (λ=0.1)
Volatility
0.05
0.04
0.03
0.02
0.01
2008
2010
2012
2014
2016
2018
2020
2022
2024
Date

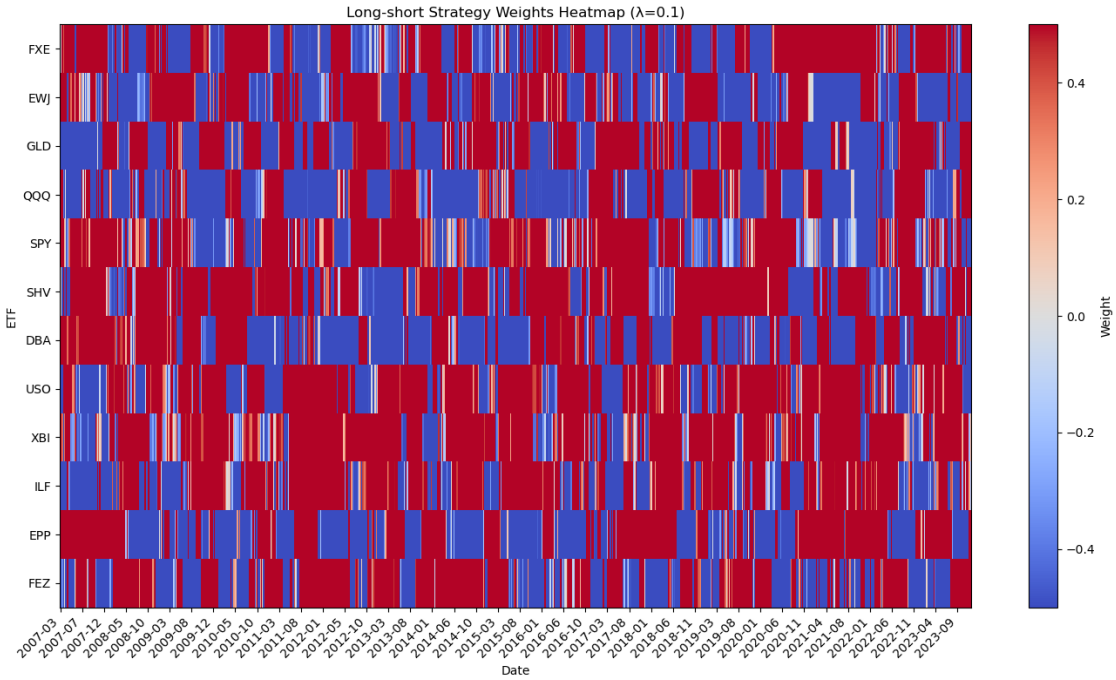

Sharpe Ratio Comparison: Long-only Strategy
Sharpe Ratio
0.05
0.04
0.03
0.02
0.01
0.00
λ=0.1
λ=0.5
λ=1.0
Lambda

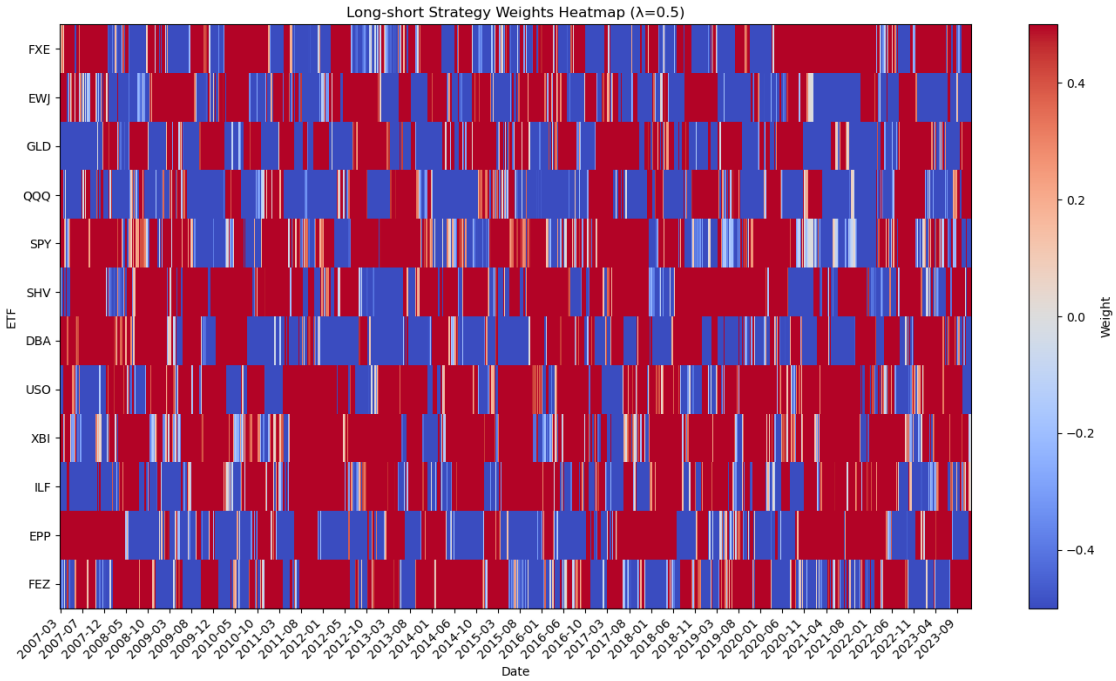

Sharpe Ratio Comparison: Long-short Strategy
Sharpe Ratio
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
λ=0.1
λ=0.5
λ=1.0
Lambda

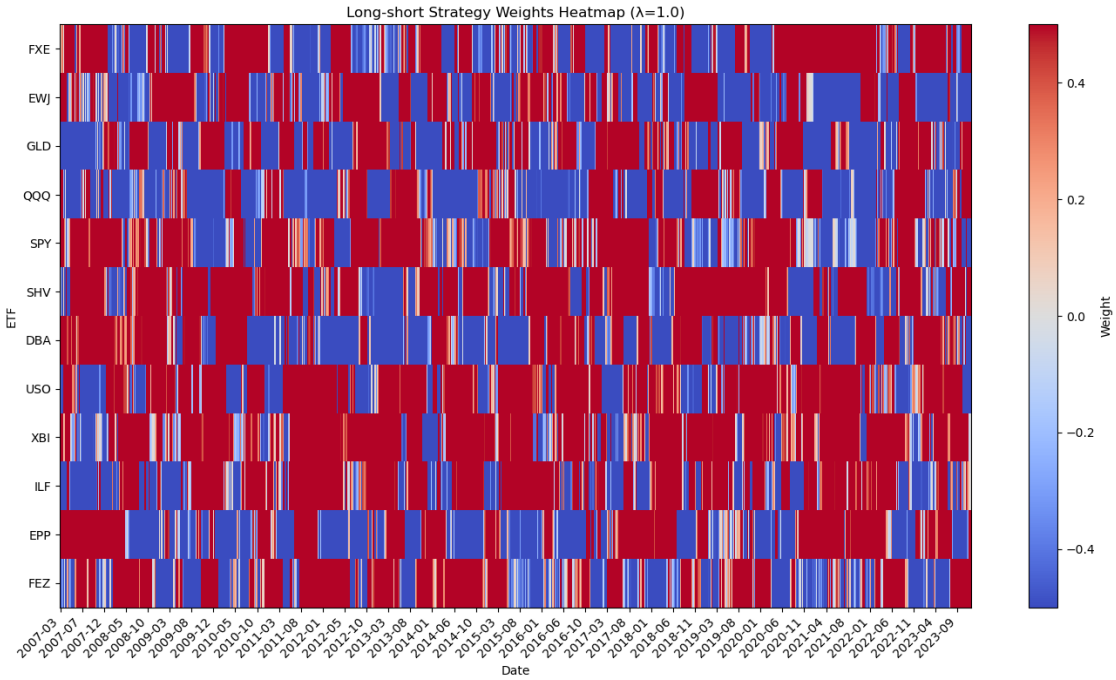

Long-only Strategy Weights Heatmap (λ=0.1)
ETF
FXE
EWJ
GLD
QQQ
SPY
SHV
DBA
USO
XBI
ILF
EPP
FEZ
Date
Weight
0.00
0.05
0.10
0.15
0.20
0.25
0.30


Long-only Strategy Weights Heatmap (λ=0.5)
ETF
FXE
EWJ
GLD
QQQ
SPY
SHV
DBA
USO
XBI
ILF
EPP
FEZ
Date
Weight
0.00
0.05
0.10
0.15
0.20
0.25
0.30


Long-only Strategy Weights Heatmap (λ=1.0)
ETF
FXE
EWJ
GLD
QQQ
SPY
SHV
DBA
USO
XBI
ILF
EPP
FEZ
Date
Weight
0.00
0.05
0.10
0.15
0.20
0.25
0.30


Long-short Strategy Weights Heatmap (λ=0.1)
FXE
EWJ
GLD
QQQ
SPY
SHV
DBA
USO
XBI
ILF
EPP
FEZ
ETF
Date
Weight
0.4
0.2
0.0
−0.2
−0.4


Long-short Strategy Weights Heatmap (λ=0.5)
FXE
EWJ
GLD
QQQ
SPY
SHV
DBA
USO
XBI
ILF
EPP
FEZ
ETF
Date
Weight
0.4
0.2
0.0
−0.2
−0.4



10 Block 9: Skewness, Kurtosis, VaR, CVaR, and Summary Comparisons

10.1 Overview:

This block calculates key statistical and risk metrics for both strategies (long-only and long-short) across all lambda values. It also generates a summary table comparing these metrics and visualizes the return distributions using histograms.

10.2 Key Metrics:

1. **Skewness**:
   - Measures the asymmetry of the return distribution.
   - Positive skewness: Distribution tails to the right (more large gains).
   - Negative skewness: Distribution tails to the left (more large losses).
2. **Kurtosis**:
   - Measures the "tailedness" of the return distribution.
   - High kurtosis: Presence of outliers (fat tails).
   - Low kurtosis: Less extreme values in the tails.
3. **Value at Risk (VaR)**:
   - 95% VaR represents the maximum expected loss over a period with 95% confidence.
   - Calculated as the 5th percentile of returns.
4. **Conditional VaR (CVaR)**:
   - 95% CVaR is the expected loss given that the loss exceeds the VaR threshold.
   - Provides a better understanding of tail risk.

10.3 Summary Table:

- Displays a comparison of the above metrics for both strategies and lambda values.

10.4 Visualizations:

1. **Histograms of Returns**:
   - Illustrates the distribution of daily returns for each strategy and lambda value.
   - Helps identify skewness, kurtosis, and outliers in the data.

10.5 Potential Improvements:

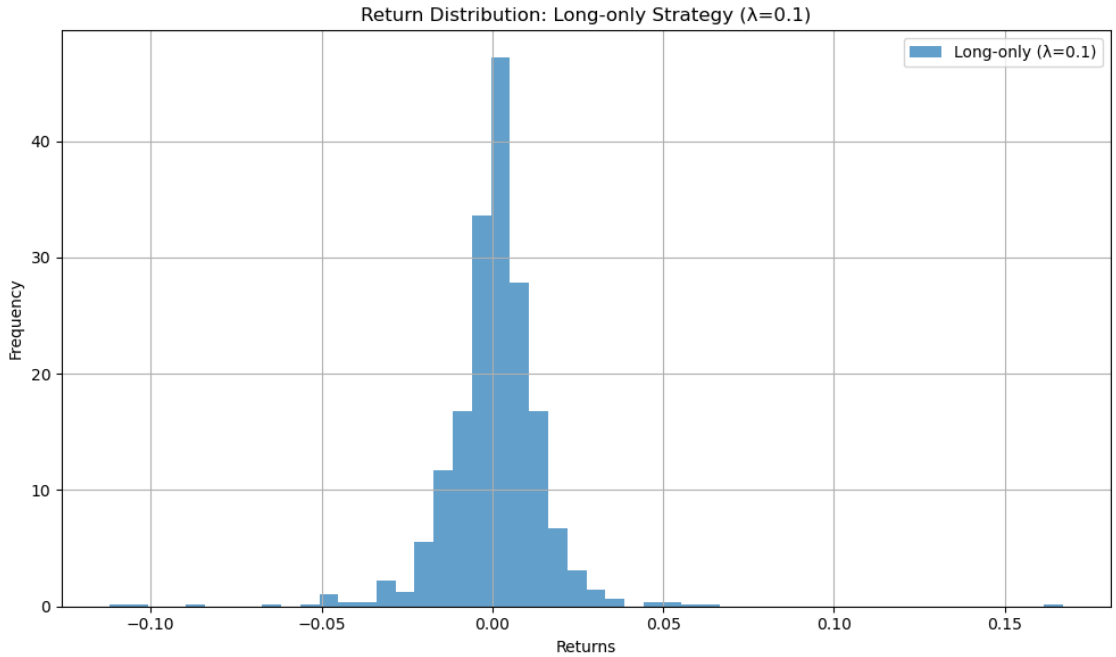
- Include additional metrics like maximum drawdown or Sharpe ratio for a broader risk-return profile.
- Extend analysis to account for transaction costs or turnover rates for better real-world applicability.

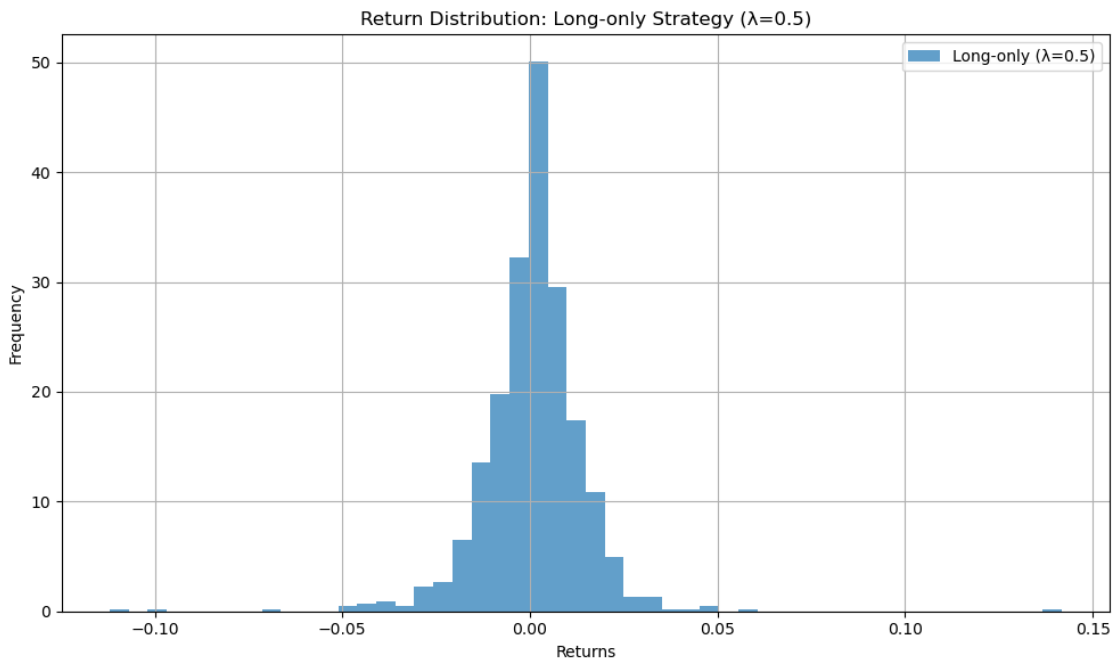
This block helps compare the performance and risk characteristics of the strategies, enabling a deeper understanding of their behavior under different risk aversion parameters (λ).

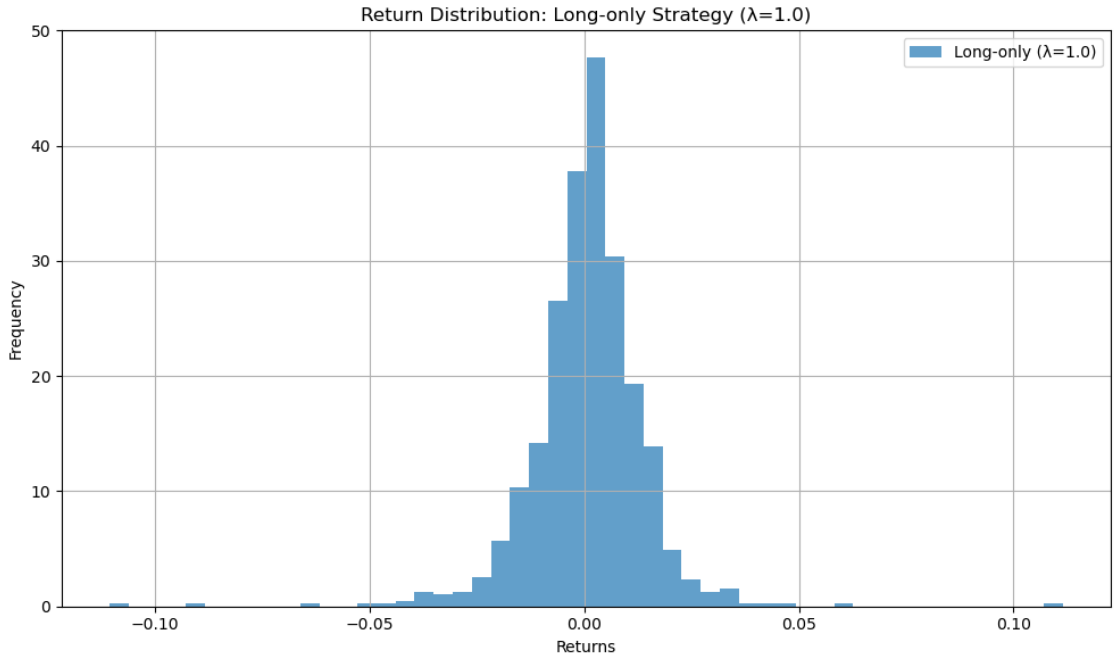
[26]:
```
def calculate_risk_metrics(returns):
    """
    Calculate skewness, kurtosis, VaR, and CVaR for a given series of returns.
    """
    skewness = returns.skew()
    kurtosis = returns.kurt()
    var_95 = np.percentile(returns, 5)  # 5% Value at Risk (VaR)
    cvar_95 = returns[returns <= var_95].mean()  # Conditional VaR (CVaR)
    return skewness, kurtosis, var_95, cvar_95

def summarize_metrics(portfolio_history, strategies, lambdas):
    """
    Summarize metrics for all strategies and lambdas in a tabular format.
    """
    summary = []
    for strategy in strategies:
        for risk_aversion in lambdas:
            history_df = portfolio_history[strategy][risk_aversion]
            returns = history_df['Portfolio Value'].pct_change().dropna()
            skewness, kurtosis, var_95, cvar_95 = calculate_risk_metrics(returns)
            summary.append({
                'Strategy': strategy,
                'Lambda': risk_aversion,
                'Skewness': skewness,
                'Kurtosis': kurtosis,
                'VaR (95%)': var_95,
                'CVaR (95%)': cvar_95
            })
    return pd.DataFrame(summary)
```

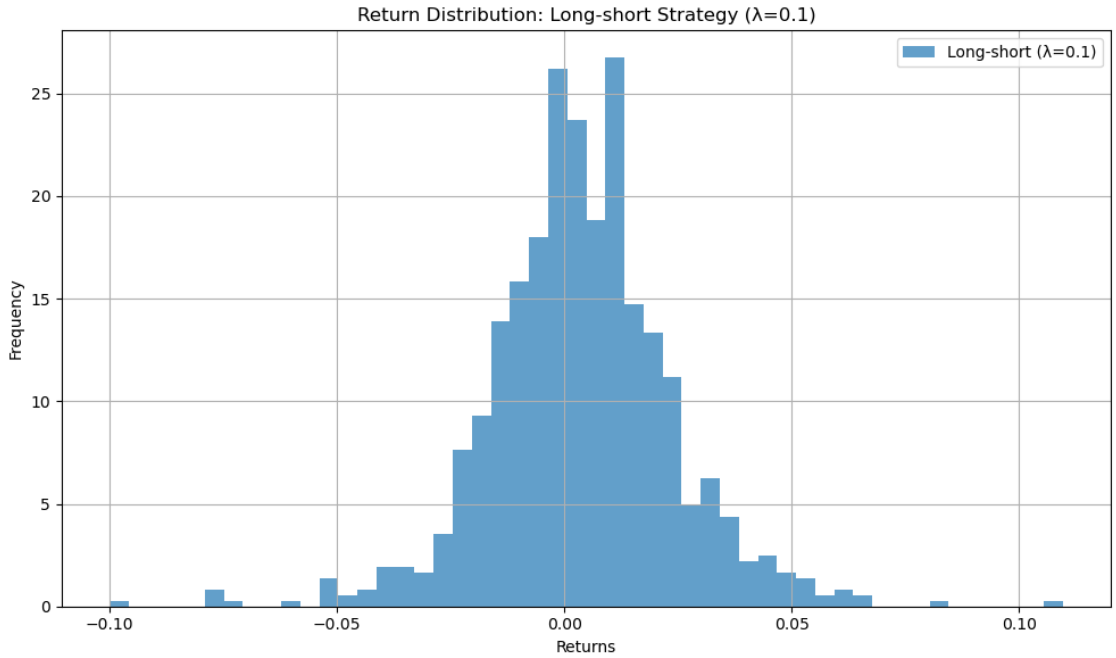
```python
def plot_return_histograms(portfolio_history, strategies, lambdas):
    """
    Plot histograms of returns for all strategies and lambdas.
    """
    for strategy in strategies:
        for risk_aversion in lambdas:
            history_df = portfolio_history[strategy][risk_aversion]
            returns = history_df['Portfolio Value'].pct_change().dropna()
            plt.figure(figsize=(10, 6))
            plt.hist(returns, bins=50, alpha=0.7, label=f'{strategy.
 ↪capitalize()} (λ={risk_aversion})', density=True)
            plt.title(f'Return Distribution: {strategy.capitalize()} Strategy␣
 ↪(λ={risk_aversion})')
            plt.xlabel('Returns')
            plt.ylabel('Frequency')
            plt.legend()
            plt.grid()
            plt.tight_layout()
            plt.show()

# Execute and display results
summary_table = summarize_metrics(portfolio_history, strategies, lambdas)
print("Summary Table:")
print(summary_table)

# Plot histograms of returns
plot_return_histograms(portfolio_history, strategies, lambdas)
```

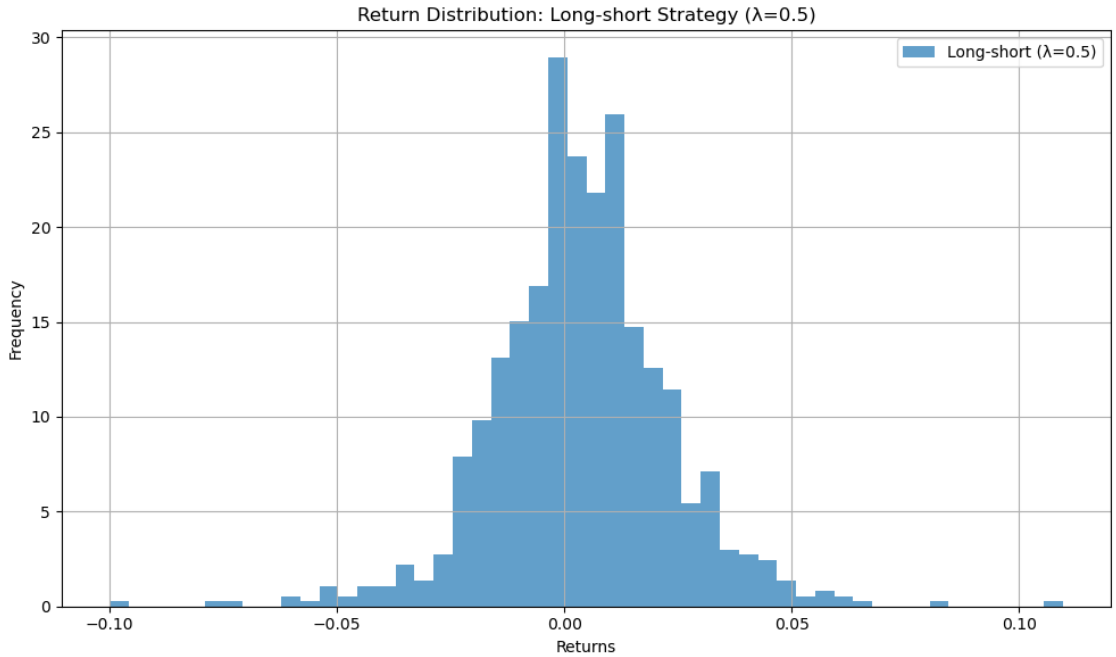
```
Summary Table:
     Strategy  Lambda  Skewness   Kurtosis  VaR (95%)  CVaR (95%)
0   long-only     0.1  0.394942  24.707862  -0.019669   -0.035471
1   long-only     0.5 -0.086196  20.060834  -0.018592   -0.032949
2   long-only     1.0 -0.625981  14.233197  -0.018592   -0.031872
3  long-short     0.1 -0.068686   2.704168  -0.026687   -0.042264
4  long-short     0.5 -0.062659   2.845584  -0.023954   -0.040537
5  long-short     1.0  0.031014   3.039099  -0.023178   -0.037838
```

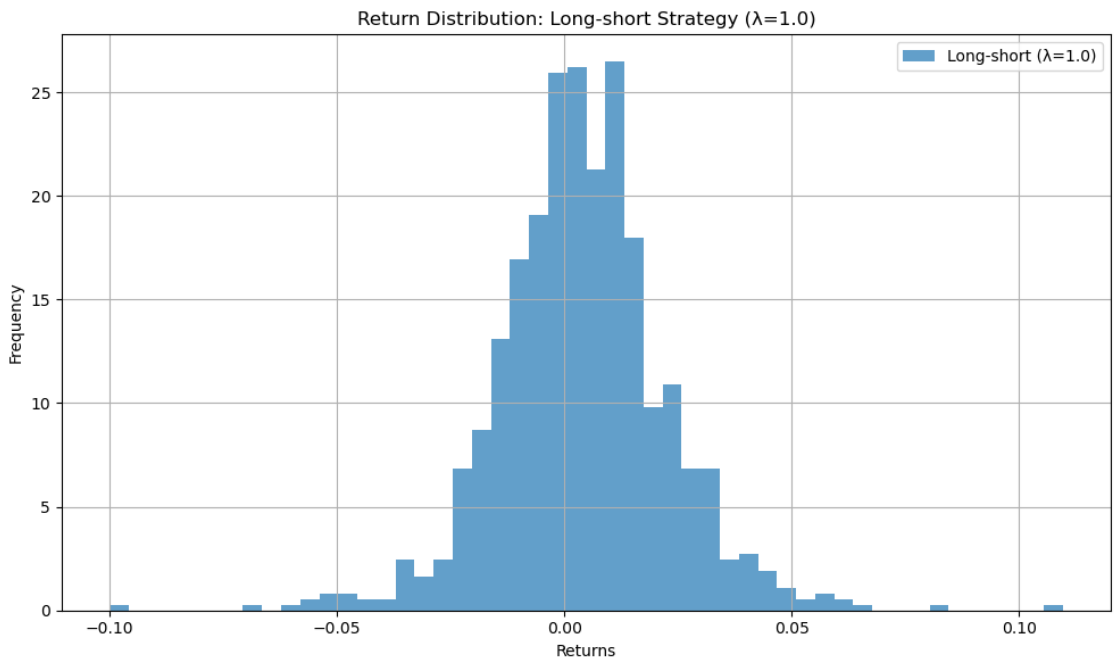

Return Distribution: Long-only Strategy (λ=0.1)
Long-only (λ=0.1)
Frequency
Returns
0
10
20
30
40
−0.10
−0.05
0.00
0.05
0.10
0.15


Return Distribution: Long-only Strategy (λ=0.5)
Long-only (λ=0.5)
Frequency
Returns
0
10
20
30
40
50
−0.10
−0.05
0.00
0.05
0.10
0.15


Return Distribution: Long-only Strategy (λ=1.0)
Long-only (λ=1.0)
Frequency
Returns
0
10
20
30
40
50
−0.10
−0.05
0.00
0.05
0.10


Return Distribution: Long-short Strategy (λ=0.1)
Long-short (λ=0.1)
Frequency
Returns
0
5
10
15
20
25
−0.10
−0.05
0.00
0.05
0.10


Return Distribution: Long-short Strategy (λ=0.5)
Long-short (λ=0.5)
Frequency
Returns
0
5
10
15
20
25
30
−0.10
−0.05
0.00
0.05
0.10


Return Distribution: Long-short Strategy (λ=1.0)
Long-short (λ=1.0)
Frequency
Returns
0
5
10
15
20
25
−0.10
−0.05
0.00
0.05
0.10

11 Block 10: Performance and Risk Reporting

11.1 Overview:

This block generates a comprehensive performance and risk report for the **long-only** and **long-short** strategies across all risk aversion levels (λ). The report compares the strategies against the SPY benchmark.

11.2 Key Metrics:

1. **Annualized Mean Return**:
   - The average return scaled to an annual basis.
2. **Geometric Return**:
   - The compounded annual growth rate (CAGR) of the portfolio.
3. **Minimum Daily Return**:
   - The worst daily return observed during the analysis period.
4. **Max Drawdown**:
   - The maximum observed drop from a peak to a trough in the cumulative return.
5. **Volatility**:
   - Annualized standard deviation of daily returns, representing portfolio risk.
6. **Sharpe Ratio**:
   - A measure of risk-adjusted return:

$$\text{Sharpe Ratio} = \frac{\text{Annualized Return}}{\text{Annualized Volatility}}$$

11.3 Outputs:

1. **Performance Summary Table**:
   - Compares the key metrics (above) for:
     - Long-only and long-short strategies at different λ values.
     - SPY benchmark.
2. **Cumulative Profit and Loss (PnL)**:
   - Plots the evolution of cumulative returns for all strategies and compares them to the SPY benchmark.
3. **Return Distribution**:
   - Histograms illustrating the distribution of daily returns for all strategies and SPY, allowing for risk-return comparison.

11.4 How to Interpret:

- **Cumulative PnL**: Analyze which strategies outperform or underperform the benchmark.
- **Return Histograms**: Assess the skewness, volatility, and frequency of extreme returns.
- **Summary Table**: Compare performance metrics such as returns, risk, and Sharpe ratios.

11.5 Potential Extensions:

- Include additional risk metrics like **Sortino Ratio** or **Treynor Ratio**.
- Account for **transaction costs** or **turnover** to evaluate practical performance.
- Extend the analysis to sub-periods (e.g., market regimes) for deeper insights.

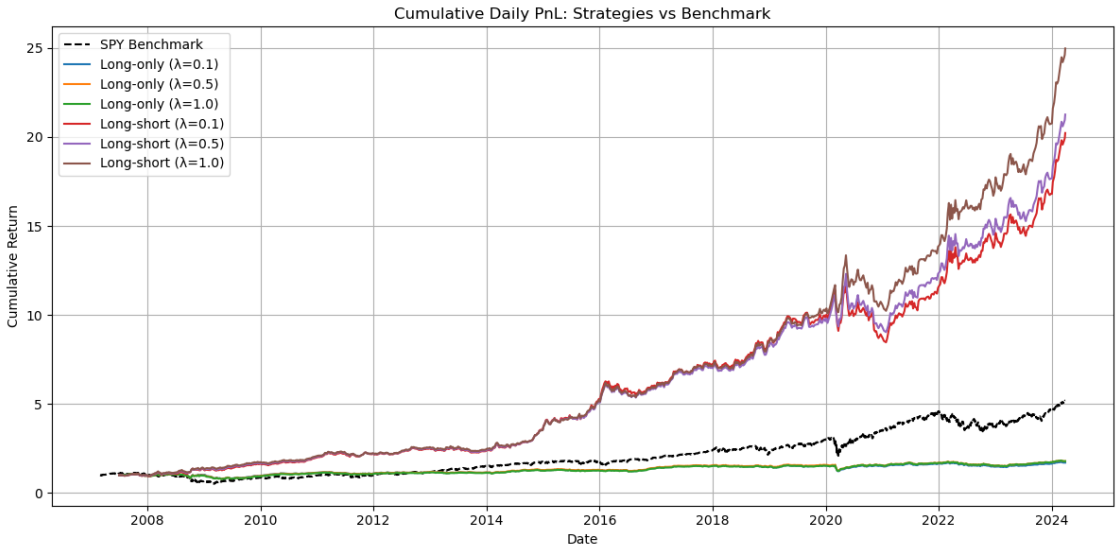
[31]:
```
def calculate_performance_metrics(history_df):
    """
    Calculate key performance metrics: mean return, geometric return, max␣
↪drawdown, volatility, and Sharpe ratio.
    """
    returns = history_df['Portfolio Value'].pct_change().dropna()
    cum_returns = (1 + returns).cumprod()

    # Metrics
    mean_return = returns.mean() * 252  # Annualized Mean Return
    geo_return = (cum_returns.iloc[-1]) ** (252 / len(returns)) - 1  #␣
↪Annualized Geometric Return
    min_daily_return = returns.min()  # Minimum Daily Return
    max_dd = (cum_returns / cum_returns.cummax() - 1).min()  # Max Drawdown
    volatility = returns.std() * np.sqrt(252)  # Annualized Volatility
    sharpe_ratio = mean_return / volatility  # Sharpe Ratio

    return {
        'Mean Return': mean_return,
        'Geometric Return': geo_return,
        'Min Daily Return': min_daily_return,
        'Max Drawdown': max_dd,
        'Volatility': volatility,
        'Sharpe Ratio': sharpe_ratio
    }

def performance_summary_table(portfolio_history, strategies, lambdas,␣
↪benchmark_returns):
    """
    Generate a summary table for comparison between strategies and SPY benchmark.
    """
    summary = []
    for strategy in strategies:
        for risk_aversion in lambdas:
            history_df = portfolio_history[strategy][risk_aversion]
            metrics = calculate_performance_metrics(history_df)
            metrics['Strategy'] = strategy
            metrics['Lambda'] = risk_aversion
            summary.append(metrics)

    # Benchmark Metrics
    spy_cum_returns = (1 + benchmark_returns).cumprod()
    spy_metrics = {
        'Strategy': 'SPY Benchmark',
        'Lambda': '-',
        'Mean Return': benchmark_returns.mean() * 252,
```

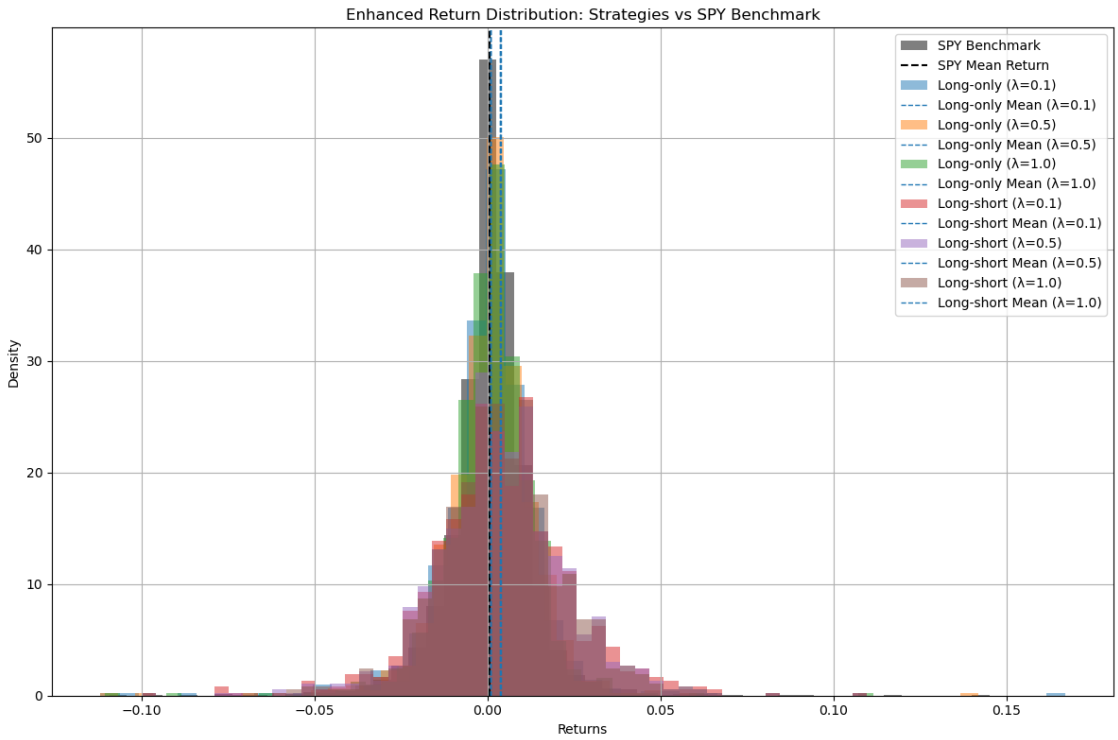
```
        'Geometric Return': (spy_cum_returns.iloc[-1]) ** (252 /␣
↪len(benchmark_returns)) - 1,
        'Min Daily Return': benchmark_returns.min(),
        'Max Drawdown': (spy_cum_returns / spy_cum_returns.cummax() - 1).min(),
        'Volatility': benchmark_returns.std() * np.sqrt(252),
        'Sharpe Ratio': (benchmark_returns.mean() * 252) / (benchmark_returns.
↪std() * np.sqrt(252))
    }
    summary.append(spy_metrics)

    # Convert to DataFrame
    summary_df = pd.DataFrame(summary)
    print("Performance Summary Table:")
    display(summary_df)

def plot_pnl_and_returns(portfolio_history, benchmark_returns, strategies,␣
↪lambdas):
    """
    Plot cumulative daily PnL for strategies and benchmark.
    Also plot the distribution of daily returns.
    """
    plt.figure(figsize=(12, 6))

    # SPY Benchmark Cumulative Returns
    spy_cum_returns = (1 + benchmark_returns).cumprod()
    plt.plot(spy_cum_returns.index, spy_cum_returns.values, label='SPY␣
↪Benchmark', linestyle='--', color='black')

    # Strategies
    for strategy in strategies:
        for risk_aversion in lambdas:
            history_df = portfolio_history[strategy][risk_aversion]
            cum_returns = history_df['Portfolio Value'] / history_df['Portfolio␣
↪Value'].iloc[0]
            plt.plot(history_df['Date'], cum_returns, label=f'{strategy.
↪capitalize()} (λ={risk_aversion})')

    plt.title("Cumulative Daily PnL: Strategies vs Benchmark")
    plt.xlabel("Date")
    plt.ylabel("Cumulative Return")
    plt.legend()
    plt.grid()
    plt.tight_layout()
    plt.show()

    # Plot distribution of daily returns
```

```
    # plt.figure(figsize=(12, 6))
    # for strategy in strategies:
    #     for risk_aversion in lambdas:
    #         history_df = portfolio_history[strategy][risk_aversion]
    #         returns = history_df['Portfolio Value'].pct_change().dropna()
    #         plt.hist(returns, bins=50, alpha=0.5, label=f'{strategy.
↪capitalize()} (λ={risk_aversion})', density=True)

    # plt.hist(benchmark_returns, bins=50, alpha=0.5, label='SPY Benchmark',␣
↪color='black', density=True)
    # plt.title("Distribution of Daily Returns")
    # plt.xlabel("Returns")
    # plt.ylabel("Frequency")
    # plt.legend()
    # plt.grid()
    # plt.tight_layout()
    # plt.show()

def plot_enhanced_return_histograms(portfolio_history, strategies, lambdas,␣
↪benchmark_returns):
    """
    Plot enhanced histograms of returns for all strategies and SPY benchmark.
    Improved visibility with density lines and formatting.
    """
    plt.figure(figsize=(12, 8))
    bins = 50

    # Plot SPY Benchmark Returns
    plt.hist(benchmark_returns, bins=bins, alpha=0.5, label='SPY Benchmark',␣
↪color='black', density=True)
    plt.axvline(benchmark_returns.mean(), color='black', linestyle='dashed',␣
↪linewidth=1.5, label='SPY Mean Return')

    # Plot Strategies
    for strategy in strategies:
        for risk_aversion in lambdas:
            history_df = portfolio_history[strategy][risk_aversion]
            returns = history_df['Portfolio Value'].pct_change().dropna()
            plt.hist(returns, bins=bins, alpha=0.5, density=True,␣
↪label=f'{strategy.capitalize()} (λ={risk_aversion})')
            plt.axvline(returns.mean(), linestyle='dashed', linewidth=1,␣
↪label=f'{strategy.capitalize()} Mean (λ={risk_aversion})')

    # Customize Plot
    plt.title("Enhanced Return Distribution: Strategies vs SPY Benchmark")
    plt.xlabel("Returns")
```

```
    plt.ylabel("Density")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

# Execute the performance reporting
benchmark_returns = daily_returns['SPY']
performance_summary_table(portfolio_history, strategies, lambdas,␣
 ↪benchmark_returns)
plot_pnl_and_returns(portfolio_history, benchmark_returns, strategies, lambdas)
plot_enhanced_return_histograms(portfolio_history, strategies, lambdas,␣
 ↪benchmark_returns)
```

```
Performance Summary Table:

   Mean Return  Geometric Return  Min Daily Return  Max Drawdown  Volatility  \
0     0.181471          0.165040         -0.112046     -0.275395    0.239800
1     0.191353          0.181194         -0.112046     -0.265654    0.222466
2     0.188765          0.180424         -0.110659     -0.271951    0.213156
3     0.919915          1.379556         -0.100033     -0.288132    0.321413
4     0.930651          1.414183         -0.100032     -0.266040    0.309487
5     0.973656          1.528942         -0.100032     -0.233891    0.297794
6     0.116648          0.101255         -0.109424     -0.551895    0.200781

   Sharpe Ratio       Strategy Lambda
0      0.756760      long-only    0.1
1      0.860144      long-only    0.5
2      0.885571      long-only    1.0
3      2.862096     long-short    0.1
4      3.007080     long-short    0.5
5      3.269563     long-short    1.0
6      0.580974  SPY Benchmark      -
```


Cumulative Daily PnL: Strategies vs Benchmark
SPY Benchmark
Long-only (λ=0.1)
Long-only (λ=0.5)
Long-only (λ=1.0)
Long-short (λ=0.1)
Long-short (λ=0.5)
Long-short (λ=1.0)
Cumulative Return
Date
0
5
10
15
20
25
2008
2010
2012
2014
2016
2018
2020
2022
2024


Enhanced Return Distribution: Strategies vs SPY Benchmark
SPY Benchmark
SPY Mean Return
Long-only (λ=0.1)
Long-only Mean (λ=0.1)
Long-only (λ=0.5)
Long-only Mean (λ=0.5)
Long-only (λ=1.0)
Long-only Mean (λ=1.0)
Long-short (λ=0.1)
Long-short Mean (λ=0.1)
Long-short (λ=0.5)
Long-short Mean (λ=0.5)
Long-short (λ=1.0)
Long-short Mean (λ=1.0)
Density
Returns
0
10
20
30
40
50
−0.10
−0.05
0.00
0.05
0.10
0.15

12 Block 11: Detailed Performance Comparison Across Lookback Periods

12.1 Overview:

This block evaluates and compares multiple performance metrics for different strategies (**long-only** and **long-short**) across various lookback periods (180, 90, and 50 days). The metrics are presented in a tabular format where:

- **Rows** represent the key performance metrics.
- **Columns** represent combinations of strategies and lookback periods.

12.2 Key Metrics:

1. **Cumulated PnL or Return**:
   - Overall portfolio return over the entire analysis period.
2. **Daily Mean Return**:
   - **Arithmetic**: The average of daily returns annualized to 252 trading days.
   - **Geometric**: The compounded annual growth rate of the portfolio.
3. **Daily Min Return**:
   - The worst single-day return observed.
4. **Max 10 Days Drawdown**:
   - The maximum loss experienced over any rolling 10-day period.
5. **Volatility**:
   - Annualized standard deviation of daily returns, measuring risk.
6. **Sharpe Ratio**:
   - Risk-adjusted return calculated as:

$$\text{Sharpe Ratio} = \frac{\text{Annualized Return}}{\text{Annualized Volatility}}$$

7. **Skewness**:
   - Measures the asymmetry of the return distribution.
8. **Kurtosis**:
   - Captures the fat-tailed nature of the return distribution.
9. **Value at Risk (VaR)**:
   - 95% VaR represents the worst expected loss at a 95% confidence level.
10. **Conditional VaR (CVaR)**:
    - Average loss occurring beyond the VaR threshold.

12.3 Outputs:

- A table that summarizes all the above metrics for:
  - **Three lookback periods**: 180 days, 90 days, and 50 days.
  - **Two strategies**: Long-only and Long-short.
- The table format has:
  - **Metrics as rows**.
  - **Strategies and lookback periods as columns**.

12.4 Insights:

- Compare how performance metrics (like Sharpe ratio, volatility, or max drawdown) vary across strategies and lookback periods.
- Assess which combination of strategy and lookback period provides optimal returns and risk-adjusted performance.

12.5 Use Cases:

- Identify the lookback period that best balances responsiveness to market changes and performance stability.
- Understand the trade-off between risk and return for long-only vs. long-short strategies.

[43]:
```
def calculate_detailed_metrics(history_df):
    """
    Calculate detailed performance metrics for a given portfolio history.
    """
    returns = history_df['Portfolio Value'].pct_change().dropna()
    cumulated_pnl = history_df['Portfolio Value'].iloc[-1] /␣
↪history_df['Portfolio Value'].iloc[0] - 1
    mean_return_arithmetic = returns.mean() * 252  # Annualized Arithmetic Mean␣
↪Return
    mean_return_geometric = (1 + returns).prod() ** (252 / len(returns)) - 1  #␣
↪Annualized Geometric Return
    min_daily_return = returns.min()
    rolling_drawdown = (history_df['Portfolio Value'] / history_df['Portfolio␣
↪Value'].cummax()) - 1
    max_10_day_drawdown = rolling_drawdown.rolling(window=10).min().min()
    volatility = returns.std() * np.sqrt(252)  # Annualized Volatility
    sharpe_ratio = mean_return_arithmetic / volatility
    skewness = returns.skew()
    kurtosis = returns.kurt()
    var_95 = np.percentile(returns, 5)  # 5% Value at Risk (VaR)
    cvar_95 = returns[returns <= var_95].mean()  # Conditional VaR (CVaR)

    return {
        'Cumulated PnL': cumulated_pnl,
        'Daily Mean Return (Arithmetic)': mean_return_arithmetic,
        'Daily Mean Return (Geometric)': mean_return_geometric,
        'Min Daily Return': min_daily_return,
        'Max 10 days Drawdown': max_10_day_drawdown,
        'Volatility': volatility,
        'Sharpe Ratio': sharpe_ratio,
        'Skewness': skewness,
        'Kurtosis': kurtosis,
        'VaR (95%)': var_95,
        'CVaR (95%)': cvar_95
    }
```

```
def compare_lookback_detailed_summary_with_spy(excess_returns, strategies,␣
↪lookback_periods, spy_returns):
    """
    Generate a summary table for multiple performance metrics across lookback␣
↪periods and strategies,
    including SPY benchmark metrics for comparison.
    """
    metrics_results = []
    lambda_fixed = 0.1  # Fixed lambda value for this analysis

    # Calculate SPY benchmark metrics
    spy_history = (1 + spy_returns).cumprod()  # Cumulative SPY values
    spy_df = pd.DataFrame({'Portfolio Value': spy_history})
    spy_metrics = calculate_detailed_metrics(spy_df)
    spy_metrics['Strategy'] = 'SPY'
    spy_metrics['Lookback'] = 'Benchmark'
    metrics_results.append(spy_metrics)

    # Initialize the progress bar
    total_iterations = len(lookback_periods) * len(strategies)
    with tqdm(total=total_iterations, desc="Processing Lookbacks and␣
↪Strategies") as pbar:
        for lookback in lookback_periods:
            for strategy in strategies:
                portfolio_value = 1
                history = []
                for date in rebalancing_dates:
                    if date not in excess_returns.index:
                        date = excess_returns.index[excess_returns.index.
↪get_loc(date, method='nearest')]

                    start_period = date - timedelta(days=lookback)
                    if start_period < excess_returns.index[0]:
                        continue

                    # Compute factor loadings and optimize portfolio
                    factor_loadings = compute_factor_loadings(
                        excess_returns.loc[start_period:date], fama_french_data.
↪loc[start_period:date]
                    )
                    expected_returns = factor_loadings[0].values
                    covariance_matrix = excess_returns.loc[start_period:date].
↪cov().values
                    beta = factor_loadings[1].values
```

```
                    # Optimize weights
                    weights = optimize_portfolio(expected_returns,␣
↪covariance_matrix, beta, strategy, lambda_fixed)
                    portfolio_return = np.dot(weights, daily_returns.loc[date].
↪values)
                    portfolio_value *= (1 + portfolio_return)
                    history.append({'Date': date, 'Portfolio Value':␣
↪portfolio_value})

                # Create a DataFrame for portfolio history
                history_df = pd.DataFrame(history)
                metrics = calculate_detailed_metrics(history_df)
                metrics['Strategy'] = strategy
                metrics['Lookback'] = lookback
                metrics_results.append(metrics)

                # Update the progress bar
                pbar.update(1)

    # Convert results to DataFrame
    metrics_df = pd.DataFrame(metrics_results)
    metrics_df.set_index(['Strategy', 'Lookback'], inplace=True)
    metrics_df = metrics_df.T  # Transpose to match the format in the screenshot

    print("Detailed Performance Metrics Including SPY Benchmark:")
    display(metrics_df)

# Define lookback periods to compare
lookback_periods = [180, 90, 50]
spy_returns = daily_returns['SPY']
compare_lookback_detailed_summary_with_spy(excess_returns, strategies,␣
↪lookback_periods, spy_returns)
```

```
Processing Lookbacks and Strategies: 100%|| 6/6 [01:50<00:00, 18.48s/it]
```

```
Detailed Performance Metrics Including SPY Benchmark:
```

Strategy	SPY	long-only	long-short	long-only	\
Lookback	Benchmark	180	180	90	
Cumulated PnL	4.251848	1.062523	25.269757	2.271969	
Daily Mean Return (Arithmetic)	0.117443	0.247183	1.002791	0.377728	
Daily Mean Return (Geometric)	0.102131	0.234792	1.591370	0.405266	
Min Daily Return	-0.109424	-0.122367	-0.086752	-0.176744	
Max 10 days Drawdown	-0.551895	-0.316909	-0.221410	-0.270584	
Volatility	0.200777	0.269752	0.313338	0.271436	
Sharpe Ratio	0.584943	0.916334	3.200349	1.391591	
Skewness	-0.065841	0.504230	0.121531	-0.954510	

Kurtosis	13.894106	16.531032	1.894677	20.299856
VaR (95%)	-0.019056	-0.022042	-0.026978	-0.023060
CVaR (95%)	-0.030964	-0.037830	-0.039369	-0.038571

Strategy	long-short	long-only	long-short
Lookback	90	50	50
Cumulated PnL	60.567259	2.037340	68.852935
Daily Mean Return (Arithmetic)	1.236713	0.360296	1.260430
Daily Mean Return (Geometric)	2.262661	0.372598	2.355195
Min Daily Return	-0.075911	-0.176872	-0.082895
Max 10 days Drawdown	-0.168383	-0.319738	-0.147252
Volatility	0.321934	0.293540	0.307876
Sharpe Ratio	3.841517	1.227417	4.093954
Skewness	0.242486	-0.468809	0.227341
Kurtosis	2.363587	20.257976	2.263184
VaR (95%)	-0.025756	-0.024714	-0.025657
CVaR (95%)	-0.039675	-0.043414	-0.036336